

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

„10 féle ember van: aki ismeri a bináris számokat, és aki nem.”

– ismeretlen szerző



```
n = 2
print(f"{n:b}_féle_ember_van: "
      "_aki_ismeri_a_bináris_számkokat ,_és_aki_nem.")
```

- Látható, hogy maga a beolvasás és a kiírás Pythonban nagyon egyszerű műveletek: egy sztringet olvasnak be vagy írnak ki, és kész.
- Beolvasáskor a kapott sztringet tudjuk feldolgozni úgy ahogy szeretnénk.
- Ha kiírásnál valami konkrét (az elemek valamivel elválasztott és valami mással lezárt sorozatánál bolyolultabb) formátumot szeretnénk megjeleníteni, azt először nekünk kell sztringként összeállítani, és a kész sztringet kell kiíratni.
- Ehhez nyújtanak segítséget a Python sztringformázó lehetőségei:
 - A `%` művelet.
 - Az *f-sztring literálok*.
 - Az `str` típus `.format()` metódusa.

A % művelet

- A `string % values` művelet segítségével a `string`-ben található `%` karakterrel kezdődő konverzió-specifikációk helyére tudjuk a `values`-ban megadott különféle értékeket különféle formában beszúrni. (Ez gyakorlatilag egy új sztring elkészítését jelenti, mert az `str` típus *immutable*.)
- A `values` lehet `tuple` vagy `dict` típusú érték.
 - Ha a `values` típusa `tuple`, akkor a konverzió-specifikációknak megfelelő számú elemet kell hogy tartalmazzon, amik sorban kerülnek felhasználásra.
 - Ha a `values` típusa `dict`, akkor az elemei kulcsok segítségével lesznek beillesztve.



A % művelet

Konverzió-specifikációk

- Egy konverzió-specifikáció általános alakja:

- `%[mapping][flags][width][.precision]type`

mapping Egy kulcs aminek az értékét be kell szúrni, ha az értékek dictionary-ben vannak megadva.

flag Több karakter, ami többféle megjelenítési jellemzőt kapcsolhat be. Elhagyható.

width Egy szám, a megjelenítési szélességét határozza meg. Elhagyható.

.precision Egy szám, a megjelenítés pontosságát adja meg. Elhagyható.

type Egy karakter, a paraméter típusát és a megjelenítés alapvető formáját határozza meg. Kötelező.

A % művelet – Konverzió-specifikációk

`%[mapping] [flags] [width] [.precision] type – [1/2]`

- Egy karakter az alábbiak közül, ami a paraméter típusát és a konverzió alapvető formátumát határozza meg:
 - `d, i, u` A paraméter egy szám, és decimális alakban kell behelyettesíteni.
 - A paraméter egy szám, és oktális alakban kell behelyettesíteni.
 - `X, x` A paraméter egy szám, és hexadecimális alakban kell behelyettesíteni.
 - `E, e` A paraméter egy szám, és normálalakban kell behelyettesíteni.
 - `F, f` A paraméter egy szám, és tizedestört alakban kell behelyettesíteni.
 - `G, g` A paraméter egy szám, és kis értékek esetén normálalakban, nagyobb értékeknél tizedestört alakban lesz behelyettesítve.

A % művelet – Konverzió-specifikációk

%[mapping] [flags] [width] [.precision] **type** – [2/2]

- c** A paraméter `int` vagy (1 hosszúságú) `str` típusú, és karakterként kell behelyettesíteni.
- r** A paramétert sztringgé konvertáljuk (a `repr()` függvény segítségével), és ezt a sztringet kell behelyettesíteni.
- s** A paramétert sztringgé konvertáljuk (az `str()` függvény segítségével), és ezt a sztringet kell behelyettesíteni.
- a** A paramétert sztringgé konvertáljuk (az `ascii()` függvény segítségével), és ezt a sztringet kell behelyettesíteni.
- %** A % karakter lesz behelyettesítve.

A % művelet – Konverzió-specifikációk

%[mapping] [flags] [width] [.precision]type

- Egy zárójelbe tett név, a paraméter dictionary egy kulcsa.
 - (key) A művelet paramétere egy dict típusú érték, amelynek a key kulcsú elemét kell beilleszteni.



A % művelet – Konverzió-specifikációk

%[mapping] [flags] [width] [.precision]type

- Nulla vagy több karakter az alábbiak közül, ami a behelyettesítés formátumát módosíthatja:

- # Számformátumra jellemző alak: tizedespont, kezdő 0o, 0x vagy 0X előtag használata.

- 0 Számok esetén a megadott szélesség balról nullákkal való feltöltése.

- A megadott szélességen belül a tartalom balra igazítása.

- szóköz Számok esetén az előjel helyének fenntartása.

- + Számok esetén az előjel kötelező kiírása.

A % művelet – Konverzió-specifikációk

`%[mapping] [flags] [width] [.precision]type`

- Egy egész érték, ami a minimális mezőszélességet határozza meg. Ha a következő paraméter ennél kevesebb behelyettesítendő karakterből áll, akkor a fennmaradt helyet szóközzel (vagy számok, jobbra igazítás és a 0 flag egyidejű használata esetén 0 karakterekkel) tölti ki. A - flag nélkül a rövidebb tartalmat a mező jobb széléhez igazítja. Ha a paraméter képe a szélességnél több karakterből áll, akkor nincs hatása.

szám Egy decimális számleírás.

- * A mezőszélesség értékét a következő (`int` típusú) argumentum határozza meg.

A % művelet – Konverzió-specifikációk

`%[mapping] [flags] [width] [.precision]type`

- Egy egész érték, ami a behelyettesítés pontosságát határozza meg. Egész szám esetén ez legalább ennyi számjegy behelyettesítését jelenti, szükség esetén balról 0 karakterekkel kiegészítve. Valós szám esetén a beszúrandó tizedesjegyek számát jelenti, sztringek esetén pedig a paraméterből behelyettesítendő karakterek maximális számát határozza meg.

`.szám` Egy decimális számleírás (a pont előttte mindenképpen szükséges).

`.*` A pontosság értékét a következő (`int` típusú) argumentum határozza meg.



A % művelet

Példák

```
>>> 'd:>%05d<>>o:>%#5o<>>x:>%-+5X<' % (42, 34, 66)
```

```
'd:>0042<>>o:>0o42<>>x:>+42<>>'
```

```
>>> 'e:>%.2e<>>f:>%#.*f<>>g:>%-+5g%%<' % (10, 123.456, 5, 123.456, 123.456)
```

```
'e:>1.23e+02<>>f:>123.45600<>>g:>+123.456%<'
```

```
>>> 'r:>%r<>>s:>%s<' % ('repr()', 'str()')
```

```
"r:>'repr()'<>>s:>str()<"
```

```
>>> 'r:>%(s)r<>>a:>%(s)a<' % {'s': 'árvíztűrő'}
```

```
"r:>'árvíztűrő'<>>a:>'\\xe1rv\\xedzt\\u0171r\\u0151'<"
```

```
>>> '>%c<>>%c<>>%-5.3s<' % (65, 'a', 'abcdef')
```

```
'>A<>>a<>>abc<>>'
```

Az *f*-sztringek (formázott sztring literálok)

- Az *f*-sztring egy `f` karakter prefixszel ellátott sztring literál (`f'...'`).
- Az *f*-sztringekben `{` és `}` karakterek közé szúrt *csere-mezők* lesznek lecserélve a bennük szereplő kifejezés aktuális értékére. Ez a csere az *f*-sztring megadásának helyén, a sztring létrehozásakor megtörténik.
- Az *f*-sztringekben a `{{` és `}}` karakterpárok jelölik a szimpla (azaz nem csere-mezőt határoló) `{` és `}` karaktereket.



Az *f*-sztringek (formázott sztring literálok)

Csere-mezők

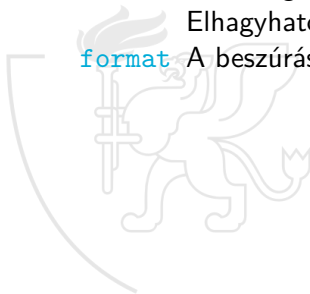
- Egy csere-mező általános alakja:
 - `{expr[=] [!conv] [:format]}`

expr Egy kifejezés.

= A kifejezés is be lesz szűrve, nem csak az értéke. Elhagyható.

conv A sztringre való konverziót ír elő és annak módját határozza meg. Elhagyható.

format A beszúrás formátumát határozza meg. Elhagyható.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

`{expr[=] [!conv] [:format]}`

- Egy (majdnem) tetszőleges python kifejezés. A kifejezés az f-sztring literál megadásának helyén, az éppen aktuális változórétékekkel lesz kiszámolva. A sztringben a csere-mező helyére a kiszámolt kifejezés értéke lesz beszúrva.



Az *f-stringek* (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [:format]}

- Ha a kifejezés után az `=` karakter szerepel, akkor nem csak a kifejezés értéke kerül behelyettesítésre, de előtte ottmarad maga a kifejezés is egy `=` jellel a végén.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

`{expr [=] [!conv] [:format]}`

- A kifejezés behelyettesítés előtti sztringgé konvertálását írja elő, és annak módját határozza meg, azaz, hogy a három lehetséges konverziós függvény közül melyik lesz használva.

!r A kifejezés a `repr()` függvény segítségével lesz sztringgé konvertálva.

!s A kifejezés az `str()` függvény segítségével lesz sztringgé konvertálva.

!a A kifejezés az `ascii()` függvény segítségével lesz sztringgé konvertálva.



Az *f-string*ek (formázott sztring literálok) – Csere-mezők

`{expr [=] [!conv] [:format]}`

- A kifejezés behelyettesítésének formátumát határozza meg.
- A formátum-specifikáció általános alakja:
 - `:[fill]align[sign][z][#][0][width][group][.precision][type]`



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [: [[fill]align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]} – [1/2]

- A behelyettesítendő kifejezés típusa és a sztringgé alakítás módja. Ha a kifejezés típusa nem az, amit várunk, hibát kapunk. Ha nem adjuk meg akkor a típusnak megfelelő alapértelmezett sztringgé alakítást használjuk.
 - b Egész szám, a bináris alakját kell behelyettesíteni.
 - c Egész szám, az ilyen kódú Unicode karaktert kell behelyettesíteni.
 - d Egész szám, decimális alakban kell behelyettesíteni. (Egészekre ez az alapértelmezett.)
 - o Egész szám, az oktális alakját kell behelyettesíteni.
 - X, x Egész szám, a hexadecimális alakját kell behelyettesíteni (nagy- illetve kisbetűkkel).

Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [: [[fill] align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]} – [2/2]

- E, e** Egy szám, amit normálalakban kell behelyettesíteni.
- F, f** Egy szám, amit tizedestört alakban kell behelyettesíteni.
- G, g** Egy szám, amit kis értékek esetén normálalakban, nagyobb értékeknél tizedestört alakban kell behelyettesíteni. (Valósakra a **g** az alapértelmezett.)
- n** Mint a **d** vagy a **g**, csak az ezres csoportosítást a gépen (globálisan) beállított formában és karakterrel végzi el.
- %** A számot százalékként írja ki az **f**-nek megfelelő formátumban, **%** jellel a végén. (1.0 = 100.0%)
- s** Egy sztring, ezt kell behelyettesíteni. (Sztringekre ez az egyetlen és alapértelmezett lehetőség.)

Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

`{expr [=] [!conv] [: [[fill] align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]}`

- Igazítás és kitöltés.

- < Az érték balra igazítása a rendelkezésre álló helyen.

- > Az érték jobbra igazítása a rendelkezésre álló helyen.

- ^ Az érték középre igazítása a rendelkezésre álló helyen.

- = Számok esetén az előjel balra, a szám jobbra igazítása a rendelkezésre álló helyen.

fill Egy karakter: igazítás esetén ez lesz az „üres helyeket” kitöltő karakter. Csak igazítással együtt adható meg.

Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [: [[fill] align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]}

- Előjel kezelése.
 - A pozitív előjel teljesen el lesz hagyva, csak a negatív lesz kitéve.
 - szóköz A pozitív előjel helye meg lesz hagyva, de csak a negatív előjel lesz kitéve.
 - + A pozitív előjel is mindenképpen ki lesz téve.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [: [[fill] align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]}

- Módosítók.

z A nullára kerekített negatív valós érték előtt elveszi az előjelet.

Számformátumra jellemző alak: tizedespont, kezdő **0b**, **0o**, **0x** vagy **0X** előtag használata.

0 Számoknál a **0=** igazítással egyenértékű.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

`{expr [=] [!conv] [: [[fill]align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]}`

- Behelyettesítési szélesség, a beszúrandó érték minimális szélessége (karakterszám). Ha az érték reprezentálásához nincs szükség ennyi karakterre, akkor az „üres helyeket” a megadott karakterrel (vagy alapesetben szóközzel) töltjük ki.

szám Decimális egész szám: a jelölt érték lesz használva.

{mező} Egész értékű beágyazott csere-mező: a kiszámolt érték lesz használva.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

{expr [=] [!conv] [: [[fill]align] [sign] [z] [#] [0] [width] [group] [.precision] [type]]}

- Számjegyek csoportosítása.

- , Decimális számleírásban (egész vagy valós típus) az ezres csoportok jelzése a , karakterrel.
- _ Decimális számleírásban (egész vagy valós típus) az ezres csoportok jelzése az _ karakterrel, illetve nem decimális számleírásban (egész típus) a négyes számjegycsoportok szeparálása az _ karakterrel.



Az *f*-sztringek (formázott sztring literálok) – Csere-mezők

`{expr [=] [!conv] [: [[fill]align][sign][z][#][0][width][group][.precision][type]]}`

- Maximális pontosság. Valós érték esetén a megjelenített tizedesjegyek száma, sztringek esetén a sztringből használt karakterek maximális száma.

`.szám` Decimális egész szám: a jelölt érték lesz használva.

`.{mező}` Egész értékű beágyazott csere-mező: a kiszámolt érték lesz használva.



Az *f*-sztringek (formázott sztring literálok)

Példák

```
>>> i, r, w, p, s = 42, 3.14, 10, 5, 'árvíztűrő'

>>> f'd:>{i**2:08,d}<_b:>{i:-#_b}<_o:>{-i:#8o}<_x:>{i:=+5X}<'
'd:>0,001,764<_b:>0b10_1010<_o:>_0o52<_x:>+_2A<'

>>> f'e:>{r:{w}.2e}<_f:>{r:#.{p}f}<_g:>{r:<+10g}<'
'e:>_3.14e+00<_f:>3.14000<_g:>+3.14_<'

>>> f'n:>{i**2:n}<_%:>{r/p=:%}<_>{i}<_>{r}<'
'n:>1764<_%:>r/p=62.8%<_>42<_>3.14<'

>>> f'r:>{"repr()"!r:^(w)s}<_s:>{s!s:.<10.5s}<'
'r:>_repr()_<_s:>árvíz.....<"

>>> f'c:>{i=:c}<_r:>{s!r}<_a:>{s!a}<'
'c:>i=*<_r:>'árvíztűrő'<_a:>'\\xe1rv\\xedzt\\u0171r\\u0151'<"
```

A `.format()` metódus

- Python-ban lehetőségünk van az `str` típus `.format(*args, **kwargs)` metódusának használatára is.
- Ez egy sztringben el tud végezni az *f-sztringeknél* látottakhoz hasonló *csere-mező* helyettesítéseket (pontosabban a `%` művelethez hasonlóan egy új sztringet fog visszaadni, amiben már a behelyettesített értékek vannak) a megadott argumentumok alapján.
- A sztringben a *csere-mező* `expr` része háromféle lehet:
 - **Üres** Ilyenkor a metódus argumentumai a megadott sorrendben helyettesítődnek be a *csere-mezők* helyére, és a sztringben csak ilyen „üres” mezők lehetnek.
 - **Szám** Ilyenkor a metódus adott sorszámú argumentumának értéke helyettesítődik be a mező helyére.
 - **Kulcs** Ilyenkor a metódus adott nevű argumentuma helyettesítődik be a mező helyére.
- A *csere-mező* további részei és azok hatása megegyezik az *f-sztringeknél* bemutatottakkal.

A .format() metódus

Példák

```
>>> 'd:>{i:,d}<_b:>{w:-#_b}<_o:>{i:#08o}<_x:>{i:=+5X}<'.format(i=2024, w=42)
'd:>2,024<_b:>0b10_1010<_o:>0o003750<_x:>+7E8<'

>>> 'e:>{0:{1}.2e}<_f:>{0:#{2}f}<_g:>{0:<+10g}<'.format(2.71, 10, 5)
'e:>_2.71e+00<_f:>2.71000<_g:>+2.71_<'

>>> 'n:>{:n}<_u%:>{: .1%}<_u>{}<_u>{}<'.format(2024, 0.25, 42, 2.71)
'n:>2024<_u%:>25.0%<_u>42<_u>2.71<'

>>> 'r:>{s!r:^{w}s}<_u s:>{s!s:.<10.5s}<'.format(s='r/s', w=10)
'r:>_ 'r/s' _ _ _ _ _ _ _ _ _ _ <_u s:>r/s.....<"

>>> 'c:>{1:c}<_u r:>{0!r}<_u a:>{0!a}<'.format('hűtő', 64)
'c:>@<_u r:>'hűtő'<_u a:>'h\\u0171t\\u0151'<"
```